Université Paris Sorbonne Nord
Institut Galilée

Master 1.
Distributed Java Programming

We will implement the merge sort algorithm[1] but executing the subtasks (recursive calls) in parallel. The algorithm is as follows:

```
mergeSort(int arr[], int left, int right){
  if (left < right){
    int mid = (left + right) / 2 ;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid+1, right);
    merge(arr, left, mid, right);
  }
}
```

where the `merge` procedure *merges* the two already sorted sub arrays.
For this exercise, use only `Thread/Runnable` and not classes/interfaces such as `ForkJoinPool`, `ExecutorService`, etc. The idea is precisely to understand how threads work and when they need to be synchornized. For simplicity, assume that the type of the array to be sorted is `int`.

1. Implement a sequential version of `merge`. Define a `main` function that sorts an array of 10.000 random generated integers.

2. Modify the previous implementation to execute, concurrently, the two recursive calls. Are the different threads competing for the same resource? Is it possible to have race conditions?

3. Compare[2] the performance of the parallel and the sequential implementation[3].

4. What if we do not execute in parallel all the recursive calls but only the first two calls? For instance, we sort in parallel the two halves of the array and then we merge them. Compare the performance of this implementation with respect to the previous ones.

[1] See for instance `https://en.wikipedia.org/wiki/Merge_sort`
[2] You can use Unix tools such as `time` or `hyperfine`.
[3] You will probably be disappointed, since there are good chances that the naive parallel implementation is not faster than the sequential one. Why?